Care Coordination Decision Guide

Use this guide with your clinical and operations teams to decide what work needs to happen, who owns it, and how to follow it through. Start with the practice's first workflow, then add care plans, case tracking, or decision support as needed.

Companion to the Care Coordination docs.

Section 1: Use Case & Participants

1.1 Which operational problems are you trying to solve first?

- Clinicians spend time on work that other team members could perform
- Work gets lost between people, teams, visits, or systems
- Staff cannot see who owns the next action or what is overdue
- Patient care spans multiple visits without a clear case history or plan
- Follow-up, protocols, or clinical recommendations are applied inconsistently
- Some combination; identify the first workflow you intend to launch

Why: establishes whether the starting point is delegation, a work queue, longitudinal tracking, or a more structured care program.

1.2 Who participates, and where does responsibility sit?

- Which clinicians, nurses, assistants, coordinators, administrative staff, patients, and caregivers perform work?

- Is work organized by individual, role, care team, service line, location, or organization?
- Who is accountable for a patient's overall care, and who owns each individual action?
- Do outside providers or partner organizations participate directly, or through exchanged messages and documents?

Why: separates ongoing care responsibility from task ownership and identifies the access and handoff boundaries.

1.3 What creates work, and where is it managed today?

- Staff action, completed intake, an incoming message, an admission/discharge event, an order or result, a missed appointment, a scheduled follow-up, or a recommendation?
- Are you replacing spreadsheets, inboxes, or another task system, or coordinating work that will remain in those systems?
- Which system owns the clinical record, the work item, and its completion status?

Why: establishes entry points, integration needs, and which system can authoritatively change each state.

1.4 How much continuity and scale do you need?

- Does work end with one action or encounter, or continue across months of care?
- Can a patient participate in several programs or cases at once?
- How many staff share a queue, how much work arrives, and are operations limited to business hours?
- What must improve to consider the first release successful?

Why: determines the need for case grouping, concurrent assignment controls, escalation, and reporting.

1.5 Does a care program require consent or service-time records?

- What program consent must be captured, reviewed, renewed, or withdrawn before enrollment or ongoing work?

- Which activities require time records, and who performed them for which patient and program?
- How will you record actual effort, corrections, overlapping activity, and the reporting period required by the program or payer?

Why: identifies program-specific consent and time-recording requirements. Task execution windows alone do not establish billable effort. Choose the time-record model and any Consent representation after confirming the program's requirements; see Consent.

Section 2: Feature Scoping

For each feature, mark Yes / No / Nice-to-have / Not sure. The § column points to the relevant deep dive. Cover 3.1 for every implementation.

#	Feature	§	Yes	No	Nice-to-have	Not sure
1	Top-of-license care: delegate eligible steps while preserving required clinical review	3.2				
2	Shared work queues, individual assignment, claiming, and reassignment	3.3				
3	Care teams or an ongoing coordinator responsible for a patient's care	3.3 3.5				
4	Multiple workflow stages, handoffs, blocked work, and completion evidence	3.4				

#	Feature	§	Yes	No	Nice-to-have	Not sure
5	Due dates, business-hour response targets, and overdue escalation	3.4				
6	Longitudinal patient case tracking across visits or concurrent programs	3.5				
7	Patient-specific care plans, goals, and progress review	3.6				
8	Reusable protocols, recurring work, and event-driven automation	3.7				
9	Clinical decision support that informs or initiates follow-up work	3.8				
10	Patient, caregiver, or external organization participation in handoffs	3.9				
11	Operational dashboards, time-in-stage reporting, and outcome measures	3.10				

Section 3: Feature Deep Dives

Cover each feature selected above. Record the chosen approach and unresolved decisions for the first workflow before expanding to additional workflows.

The approaches below name recommended resources and fields. Local workflow stages, eligibility checks, and automation are application conventions that you implement; storing these resources does not execute those rules automatically.

3.1 Work Items & Clinical Context

Decide what a work item represents and how it relates to the clinical record.

- What is the smallest unit of work that needs its own owner, status, and completion criteria?
- Is each item about a patient, a particular encounter, or practice administration?
- What is being acted on: a referral, result, questionnaire, message thread, or another resource?
- Does one person perform the whole activity, or do independently assigned steps need separate work items?

Situation	Approach
A follow-up action needs tracking	Use `Task` for the work. Link `Task.focus` to the resource being acted on and `Task.for` to the patient when applicable.
Work arises from a particular visit	Use `Task.encounter` for encounter context where appropriate; patient-level work can exist without an encounter.
Several steps have different owners or completion criteria	Use separate Tasks. Use `Task.partOf` when a parent/subtask relationship is useful; start with a shallow hierarchy.

Situation	Approach
A clinical order already exists	Keep the order, such as a `ServiceRequest`, and point `Task.focus` to it. Completing a preparation or scheduling Task should not complete the order; update the order only when its own fulfillment criteria are met.
Non-patient administrative work	Use `Task.code` for the work type and `owner` for the responsible person or organization. Omit `for` when there is no beneficiary; use `focus` only when an actual resource is being acted on. Scope access by organization or role.

See Tasks and Work Queues.

3.2 Top-of-License Care & Delegation

Decide which steps each role can perform and where clinical judgment or review is required.

- Which steps currently reach a clinician that another qualified team member could prepare or complete?
- Which actions require clinical judgment, sign-off, supervision, or escalation?
- Do eligibility rules depend on credentials, specialty, service line, jurisdiction, or the patient's program?
- Who maintains those rules, and what happens when no eligible person is available?

Situation	Approach
Work can be routed by role	Use `Task.performerType` to describe the required role and `Task.owner` for the accountable assignee or group. Role labels alone do not establish eligibility.
Preparation and clinical review are separate responsibilities	Create separate preparation and review Tasks. Reference the prepared artifact in `Task.output.valueReference`, then supply it through the review Task's `input.valueReference`. Each Task has its own owner and completion criteria.
Eligibility varies by service or jurisdiction	Use `PractitionerRole.healthcareService` for the service and `PractitionerRole.location` for licensed jurisdictions. Evaluate role, credentials, and current eligibility in the controlled assignment or booking path before setting `Task.owner`.
No eligible assignee is available	Set `Task.status` to `on-hold`, record the reason in `statusReason`, and assign `owner` to an escalation team or coordinator. Use a local `businessStatus` to make these items searchable as an exception queue.

Define the role/action matrix with the practice's clinical and operational owners. Enforce data access using Access Policies. For booking, see State-by-State Licensure; scheduling operations do not enforce licensure themselves.

3.3 Queues, Ownership & Care Teams

Decide how staff discover work, take responsibility, and hand it to someone else.

- Is work assigned directly, claimed from a shared queue, or distributed automatically?

- Which queue dimensions matter: role, service, location, urgency, or workflow stage?
- Can two staff members attempt to claim the same item? Who resolves conflicts?
- Does the patient have an ongoing care team or coordinator distinct from the person handling today's task?
- How do absence, workload, and shift changes affect assignment?
- How do staff discover new or escalated work: a live queue, an outbound notification, or both? See assignee notifications.

Situation	Approach
Direct individual assignment	Set `Task.owner` to the responsible person or practitioner role.
Shared queue	Build a filtered view of Tasks using work type (`code`), lifecycle (`status`), local stage (`businessStatus`), and required role (`performerType`). Use a group `owner` for team accountability, or leave it unset for an unclaimed role pool.
Staff claim work concurrently	Update `Task.owner` using `If-Match` with the version read. On `412 Precondition Failed`, reload and re-evaluate the claim. Do not retry by overwriting the new owner.
Ongoing patient care team	Set `CareTeam.subject` to the patient; record members, roles, and participation periods in `participant`. Link the team through `CarePlan.careTeam` or `EpisodeOfCare.team` as appropriate. Tasks retain their own owners.

Situation	Approach
Reassignment or cross-coverage	Update `Task.owner` for the same work item and retain resource history. Add `Provenance` targeting the Task when a structured actor and transfer reason are needed. A separately accountable handoff step gets its own Task.

Care team membership and task assignment describe responsibility; neither automatically grants access to related patient data.

See Message Response Tracking and Routing, Version Checking, and Resource History.

3.4 Lifecycle, Handoffs & Deadlines

Decide what each stage means and what evidence allows work to move forward.

- Which stages do staff need to operate on, and which are useful only for reporting?
- What counts as ready, blocked, complete, cancelled, or entered in error?
- Does handing off work require acknowledgment from the next owner?
- Which deadlines use elapsed time versus business hours? Does waiting for a patient or outside party pause the clock?
- Who receives overdue work, and how are repeated reminders prevented?

Situation	Approach
Standard lifecycle plus local workflow stages	Use `Task.status` for lifecycle, `businessStatus` for local stages, and `statusReason` for why work is blocked or cancelled.

Situation	Approach
Completion requires a resulting artifact	Use `Task.output.valueReference` for the result, such as a `QuestionnaireResponse`, `DocumentReference`, or response `Communication`. Validate the required output before setting `status` to `completed`; record actual completion in `executionPeriod.end`.
Several independently tracked handoffs	On completion of the prerequisite Task, have the application or Bot conditionally create the next Task with its owner and required inputs. Use `partOf` for hierarchy; implement transitions through the handoff workflow.
Due dates or response targets	Use `restriction.period.end` for the deadline and `executionPeriod` for actual work. Follow the planned-date convention for general queues and exchange. Calculate business-hour deadlines and paused intervals in application or Bot logic.
Time in each stage matters	Derive stage changes from Task resource history; add `Provenance` when actor, reason, or effective transition time needs explicit recording. Build reporting intervals from those changes. `Task.lastModified` alone is insufficient.

See the Task guide for status and timing fields. Queue ordering, deadline calculations, and escalation require application or Bot logic.

3.5 Longitudinal Patient Case Tracking

Decide whether care needs an identity and lifecycle that spans multiple visits.

- What defines a case: a condition, a treatment course, a service program, or a period of responsibility?
- What starts, pauses, transfers, and ends that case?
- Can a patient have concurrent cases, including more than one addressing the same condition?
- Can one encounter contribute to multiple cases?
- Which organization, coordinator, or care team is responsible over time?

Situation	Approach
Independent tasks or a single visit are sufficient	Keep patient and encounter context; add case grouping when a distinct longitudinal lifecycle is needed.
Care spans multiple visits under a defined responsibility	Use `EpisodeOfCare` with `patient`, `status`, `period`, and `managingOrganization`. Use `careManager` and `team` for ongoing responsibility, and link visits through `Encounter.episodeOfCare`.
Concurrent programs or treatment courses	Give each episode its own identity and lifecycle; an Encounter can reference more than one episode.
A case also needs a clinical plan	Follow the longitudinal guide's convention: reference the `EpisodeOfCare` in `CarePlan.supportingInfo`. This is an explicit supporting-context link, not a dedicated FHIR case-membership field; agree whether a plan may span multiple episodes.

Situation	Approach
Several cases share a diagnosis	Keep diagnoses in `EpisodeOfCare.diagnosis.condition` and `CarePlan.addresses`. When finding a case's plans, verify their explicit `supportingInfo` references; a patient or condition search alone can return plans for other cases.

See Longitudinal Patient Case Tracking.

3.6 Care Plans, Goals & Progress

Decide whether the team needs a coordinated clinical plan in addition to its work queue.

- Do staff need to see agreed goals and planned activities together for a patient?
- Who authors, approves, reviews, and changes the plan? Does the patient participate?
- How is goal progress assessed, and which observations or assessments support it?
- When a plan changes, what happens to already assigned work?
- What closes the plan, and can individual goals or activities finish at different times?

Situation	Approach
A patient-specific plan is needed	Set `CarePlan.subject` to the patient, `addresses` to the relevant Conditions, `goal` to Goals, and `activity.reference` to planned requests or Tasks. Use `careTeam` for the people responsible for the plan.

Situation	Approach
Goals need independent tracking	Reference each `Goal` through `CarePlan.goal`. Use `Goal.target` for the measure, target value, and due date; `achievementStatus` for progress; and `outcomeReference` for supporting Observations. Keep goal lifecycle separate from achievement.
Planned activities create staff work	For directly authored work, reference the Task in `CarePlan.activity.reference` and the authorizing plan in `Task.basedOn`; keep `Task.focus` for the item being acted on. Assess goal achievement separately from Task completion.
Plans change during treatment	Update the existing CarePlan for routine revisions. For a replacement plan, use `CarePlan.replaces` and close the prior plan appropriately. Explicitly retain, cancel, or replace outstanding Tasks; changing a plan does not migrate them automatically.

See Care Plans and Goals and Clinical Protocols for the distinction between a patient's plan and a reusable protocol.

3.7 Protocols, Recurring Work & Automation

Decide which patterns should repeat and which decisions remain with people.

- Is work created manually, from events, on a schedule, or from a reusable protocol?
- Which steps repeat for every patient, and which depend on prior outcomes?

- Are there dependencies, delays, repeated attempts, or stop conditions?
- When a protocol changes, do existing patients continue their current version or migrate?
- How are failed runs, duplicate events, and missed follow-ups detected and recovered?

Situation	Approach
Staff initiate occasional work	Create a Task with `code`, `status`, `intent`, patient context where applicable, and an owner or required role. Add `focus` for the resource being acted on.
A reusable clinical protocol is useful	Use `PlanDefinition` and `ActivityDefinition`. Medplum's `$apply` creates a CarePlan whose activity references a RequestGroup with Task actions. Check generated resources and supported behavior against the workflow before adopting it.
A resource event should create or advance work	Use tightly scoped Subscriptions and Bots. Select create/update events and transition criteria where needed to avoid triggering on unrelated edits.
Recurring or overdue work	Use a scheduled Bot to create one Task per due occurrence. Put a stable source/step/occurrence key in `Task.identifier`; stop creating work when the source plan or case is no longer eligible. For overdue work, update or escalate the existing Task.

Situation	Approach
Events may be processed more than once	Conditionally create generated Tasks by `identifier`, for example with `createResourceIfNoneExist`. Reprocessing the same event must reuse the same key. Track and retry external delivery separately; avoiding duplicate Tasks does not make an external send idempotent.
Conditional or dependent workflow steps	Use scoped transition Subscriptions and Bots to evaluate prerequisites and create or release the next Task. Implement the dependency rules in the application or Bot.

See Clinical Protocols, Subscription Extensions, Cron Jobs for Bots, and Working with FHIR Data.

3.8 Clinical Decision Support & Follow-Through

Decide how clinical guidance enters the workflow and whether it creates an obligation to act.

- Do clinicians need contextual reference material, rules-based recommendations, or predictive outputs?
- When should guidance appear: chart review, order entry, assessment review, or background monitoring?
- Is the output informational, does it require acknowledgment, or should accepted recommendations create work?
- Who can accept, dismiss, or override a recommendation, and what reason or evidence should be recorded?
- Who maintains the knowledge or model, and how will you evaluate usefulness, missed follow-up, and excessive alerts?

Situation	Approach
Guidance is needed during a clinical interaction	Configure a Bot's `cdsService` for the hook and prefetch requirements; have the client invoke it at the chosen workflow point and display returned cards. A card does not itself create a persistent work item.
A recommendation needs review and follow-up	Use a review Task with patient context and the relevant evidence in `focus` or `input`. Record the disposition in locally defined `businessStatus` values and the rationale in `note`. Reference any resulting clinical request in `output`; track its fulfillment with a separate Task.
Background rules identify patients for review	Have a Bot conditionally create a review Task with `for`, `owner` or `performerType`, and an `identifier` derived from the rule and triggering event or review period. Repeated evaluations should reuse that work item until a new review is warranted.
An external CDS Hooks service is used	Map the supported hook's context and prefetch to the service request, then render its cards in the client. Implement timeout and failure handling there; create a fallback review Task only when the workflow requires human follow-up.

See Clinical Decision Support for broader CDS considerations and CDS Hooks for the integration contract. Define clinical review and applicable governance requirements alongside the workflow design.

If every evaluation must be retained, including informational results with no follow-up, choose that persistence model separately. A review Task records work and its disposition; it is not a complete CDS evaluation record. See the GuidanceResponse design choice.

3.9 Participation, Access & External Handoffs

Decide what each participant can see and how work crosses organizational boundaries.

- Do patients or caregivers perform assigned actions, see progress, or only receive updates?
- Which notes and documents are internal, patient-visible, or intended for an external recipient?
- After reassignment or discharge, who retains access to the record?
- Do external partners update shared records, exchange structured data, or return documents and messages?
- How are replies matched to the original request, and who handles unmatched responses?

Situation	Approach
Patients or caregivers complete assigned work	Set `owner` to Patient or RelatedPerson and `for` to the patient. For forms, focus on the Questionnaire and return the QuestionnaireResponse in Task output; give staff review a separate Task. See patient work lists.
Participants need different visibility	Enforce access through `AccessPolicy` and `ProjectMembership`; define permissions for each relevant resource type.

Situation	Approach
Patient-related resources are linked together	Populate each resource's own patient-compartment references. A link to a Task, CarePlan, or message thread does not by itself propagate patient access.
Work includes messaging	Point `Task.focus` to the `Communication` thread header and `Task.for` to the patient. Reference the resolving message in `Task.output.valueReference`. Complete the Task only when the response satisfies the workflow's completion criteria.
Documents support a handoff	Store file content in `Binary` and describe it with `DocumentReference`. Use an attachment URL such as `Binary/{id}` and set the Binary's `securityContext` appropriately; do not embed base64 in `Attachment.data`.
A referral transfers a request for services	Use the Referrals Decision Guide for capture, transmission, receiving, and closure. Connect its handoffs to the shared ownership and timing rules agreed here.

Use the Access Control Decision Guide, Messaging Decision Guide, and Binary Data docs for the detailed designs. Scheduling and billing decisions should likewise link to their respective workflow documentation.

3.10 Operational Reporting & Closure

Decide what the practice needs to measure and what each completion signal means.

- Which daily views do staff need: unassigned work, due work, blocked items, workload by team, or patients needing follow-up?
- Which measures demonstrate improvement: turnaround time, fewer handoffs, appropriate delegation, successful follow-up, or goal progress?
- What timestamps and event history support those measures, including reopened work and paused clocks?
- Does completing a Task also complete an activity, referral, plan, or case? Under what conditions?

Situation	Approach
Staff need actionable work lists	Query Tasks by `owner`, `status`, `code`, and `business-status`, within the caller's access policy. Use `owner:missing=true` for unowned pools; group-owned queues require the group's owner reference instead.
Managers need aggregate or historical metrics	Build a reporting table with Task ID, previous/new stage or owner, transition time, actor, and reason, derived from history and any explicit Provenance. Calculate stage durations and paused intervals from this event sequence. Store the derived intervals in the reporting system.
Care outcomes matter	Measure goal achievement through `Goal.achievementStatus`, targets, and the Observations in `outcomeReference`. Report those alongside completed Tasks; Task completion alone does not demonstrate an improved clinical outcome.

Situation	Approach
Work, plans, and cases close independently	Update each applicable lifecycle explicitly: `Task.status`, `CarePlan.status`, `Goal.lifecycleStatus`, and `EpisodeOfCare.status`. Before closing a case, resolve or reassign its open work; no automatic cascade should be assumed.

Decisions to Record Before Implementation

- The first workflow, its entry event, and its source of truth
- Work items, clinical context, and any longitudinal case or care-plan model
- Eligible roles, accountable owners, review steps, and exception handling
- Lifecycle stages, completion evidence, deadlines, and escalation rules
- Automation and CDS behavior, including human review and failure recovery
- Access boundaries, external handoffs, and the measures of success